

Dynamic Surge Pricing

Dynamic Surge Pricing is a pricing method where the price of a product or service **changes** automatically based on current demand and availability.

DEMAND_SURGE
WEATHER_SURGE
EVENT_SURGE
CAPACITY_SURGE
TRAFFIC_SURGE

1. Core Concept

1.1. Demand:

Demand represents the number of delivery requests requiring fulfillment.

Example:

- ordersCreated
- ordersPending
- ordersWaitingForDriver
- ordersBeingPrepared

Not every order should necessarily count equally.

1.2. Supply:

Supply represents available drivers.

But:

Total Drivers $\neq$ Available Drivers

For surge pricing we care about drivers who can actually accept work.

1.3. A simple surge formula:

Surge = Demand / (Supply + 1).

+1 = A small adjustment to avoid dividing by zero when supply is 0, and to keep the number from becoming undefined.

Example 1: High demand, low supply

Demand = 100 customers
Supply = 20 drivers

Surge = $100 / (20 + 1)$
Surge = $100 / 21 \approx 4.76$
So the calculated surge score is **4.76**

Example 2: Low demand, high supply

Demand = 20 customers
Supply = 100 drivers

Surge = $20 / (100 + 1)$
Surge ≈ 0.20
The surge score is much lower because there are plenty of drivers relative to customers.

1.4. Demand-to-Supply Ratio

A basic model:

pressure = demand / availableDrivers

Example:

Demand = 100, Drivers = 100 $\Rightarrow$ Pressure = 1.0
Demand = 100, Drivers = 50 $\Rightarrow$ Pressure = 2.0

Higher pressure generally indicates greater need for surge.

1.5. Effective Driver Supply

Not every driver should count as fully available.

Suppose: 10 drivers

But:

5 are delivering

2 are going offline

1 is far away

2 are available

Effective supply may be: **2**

or perhaps: **2 + partially available drivers**

Depending on the business model. A better concept is: effectiveSupply
rather than simply: **driverCount**

1.6. Driver Capacity

A driver currently finishing a delivery may become available soon.

Therefore we can classify:

Immediate Supply

Near-Term Supply

Unavailable Supply

Example:

Available now = 10

ETA to availability < 10 min = 5

Busy = 20

2. What Are We Building?

We want to dynamically calculate delivery pricing based on:

**Demand + Driver Supply + Geographic Location +
Time + Traffic + ETA + Weather + Events + Operational Capacity**

Instead of having:

```
Hyderabad
|
+----- Surge = 1.2x
```

We want:

```
Hyderabad
|
+----- Polygon A → 1.0x
|
+----- Polygon B → 1.4x
|
+----- Polygon C → 1.8x
|
+----- Polygon D → 1.1x
```

Therefore two customers in the same city can receive different surge pricing.

3. Why Static Pricing Is Not Enough

Polygon A : Demand = High | Driver Supply = Low

Pricing engine: Polygon + Demand + Driver Supply + Traffic $\Rightarrow$ Surge Multiplier

Demand can change rapidly.

12:00 PM Orders = 100 Drivers = 100 Surge = 1.0x	12:30 PM Orders = 200 Drivers = 80 Surge = 1.5x	1:30 PM Orders = 70 Drivers = 120 Surge = 1.0x
---	--	---

Therefore pricing must react to real-time operational conditions.

4. Demand Aggregation Per Polygon

When an order is created:

```
Order
|
v
Customer Location
|
v
Polygon
|
v
Increment Demand
```

5. Supply Aggregation Per Polygon

When a driver becomes available:

```
Driver
|
v
GPS Location
|
v
Polygon
|
v
Increment Available Supply
```

If the driver moves: Polygon A → Polygon B
then:

```
Polygon A supply -= 1
Polygon B supply += 1
```

6. Polygon Demand/Supply Snapshot

Each polygon can maintain:

```
{
  "polygonId": "P100",
  "pendingOrders": 25,
  "availableDrivers": 10,
  "activeDrivers": 18,
  "averageEtaMinutes": 14,
  "trafficScore": 0.72
}
```

This becomes the input to the surge engine.

7. Basic Surge Multiplier

A simple initial model:

pressure = demand / supply

Then map pressure to multiplier.

Example:

pressure < 1.0 → 1.0x

pressure 1.0 - 1.5 → 1.1x

pressure 1.5 - 2.0 → 1.3x

pressure 2.0 - 3.0 → 1.5x

pressure > 3.0 → 2.0x

The exact thresholds are business-configurable.

8. Surge Tiers

A practical configuration:

NORMAL	1.0x
LOW	1.1x
MEDIUM	1.3x
HIGH	1.5x
VERY_HIGH	1.8x

```
enum SurgeLevel {  
    NORMAL,  
    LOW,  
    MEDIUM,  
    HIGH,  
    VERY_HIGH  
}
```

9. Minimum and Maximum Surge

Never allow the formula to produce unlimited prices.

For example:

Then:

```
minimumMultiplier = 1.0    multiplier = clamp(calculatedMultiplier, minMultiplier,maxMultiplier);  
maximumMultiplier = 2.0    This protects the pricing system from extreme values.
```

10. Smoothing Surge Changes

Without smoothing 1.2x / 1.5x / 1.1x / 1.7x / 1.0x could occur within minutes.

Instead: **Use gradual transitions.**

11. Hysteresis

Suppose surge activates at: `pressure >= 1.5` Do not immediately deactivate it at: `pressure < 1.5`

Instead:

Activate: `pressure >= 1.5`

Deactivate: `pressure <= 1.3`

This prevents rapid on/off oscillation.

12. Time-Based Surge

Demand varies by time.

Example:

Lunch 12:00 → 2:00 PM

Dinner 7:00 → 10:00 PM

A polygon may have a higher baseline during these periods.

For example:

baseMultiplier = 1.1 during peak hours.

13. Traffic-Aware Surge

Traffic affects driver productivity.

Example:

Normal traffic → 1.0

Heavy traffic → 1.2

Traffic can therefore contribute to the pressure score.

However, avoid simply multiplying every signal independently because this can produce excessive prices.

Instead normalize signals into a common score.

13. ETA-Aware Surge

Suppose:

Polygon A: Average pickup ETA = 5 min

vs

Polygon B: Average pickup ETA = 25 min

Polygon B has significantly higher operational pressure.

ETA can therefore become a useful surge feature.

14. Weather-Aware Surge

Weather can affect:

- driver availability
- travel time
- order volume

For example: Heavy rain may cause:

fewer drivers + longer travel time + higher demand

This should increase operational pressure only when the actual data supports it.

15. Event-Aware Surge

Large events can create localized demand.

- stadium
- concert
- festival
- shopping mall
- airport

Instead of increasing prices across the entire city:

Event Polygon

|

v

Higher localized pressure

16. Polygon Pressure Score

Instead of using only: `demand / supply` we can create:

```
pressureScore =  
    demandPressure  
    + supplyPressure  
    + etaPressure  
    + trafficPressure  
    + timePressure
```

Normalize each component to:
`0.0 → 1.0`

17. Dynamic Surge Score

Example

```
demandScore = 0.80  
supplyScore = 0.70  
etaScore = 0.60  
trafficScore = 0.50  
timeScore = 0.30  
Weighted surgeScore =  
    0.40 × demandScore  
    + 0.30 × supplyScore  
    + 0.15 × etaScore  
    + 0.10 × trafficScore  
    + 0.05 × timeScore
```

18. Weighted Surge Model

The weights should be configurable:

```
class SurgeWeights {  
    double demandWeight;  
    double supplyWeight;  
    double etaWeight;  
    double trafficWeight;  
    double timeWeight;  
}
```

19. Surge Price Calculation

Suppose

```
baseDeliveryFee = ₹40  
surgeMultiplier = 1.5
```

Then:

surgedFee = ₹40 × 1.5 = ₹60

Additional pricing components may then be added according to the platform's pricing policy.

20. Delivery Fee Calculation

A complete delivery price could conceptually be:

```
deliveryFee =  
    baseFee  
    + distanceFee  
    + timeFee  
    + surgeAdjustment  
    + applicableOtherCharges
```

For example:

```
baseFee      = ₹20  
distanceFee  = ₹15  
surgeFee     = ₹20
```

total = ₹55

The exact commercial formula should remain configurable.

21. Different Surge Types

The architecture can support:

DEMAND_SURGE
WEATHER_SURGE
EVENT_SURGE
CAPACITY_SURGE
TRAFFIC_SURGE

But these should feed into a common pricing engine rather than each independently multiplying the final price.

Prefer:

Signals → Pressure Score → Surge Multiplier

Instead: $1.2 \times 1.3 \times 1.4 \times 1.2$

which can quickly become unreasonable.

22. Polygon Pricing Policy

Each polygon can have configuration:

```
class PolygonPricingPolicy {
    UUID polygonId;
    BigDecimal minimumFee;
    BigDecimal maximumFee;
    double maximumSurgeMultiplier;
    boolean surgeEnabled;
    double activationThreshold;
    double deactivationThreshold;
}
```

23. Polygon Configuration

Example:

```
Polygon A

surgeEnabled = true
maxMultiplier = 1.5
activationThreshold = 1.5
deactivationThreshold = 1.3
```

This allows different operational policies

24. Surge Calculation Engine

```
public interface SurgePricingService {
    SurgeResult calculate(
        SurgeRequest request
    );
}

@Service
public class DynamicSurgePricingService
    implements SurgePricingService {

    // implementation
}
```

This keeps the pricing logic replaceable.

```
@Service
public class SurgeCalculationEngine {
    public SurgeResult
    calculate(PolygonState state) {
        // calculate pressure
        // calculate surge score
        // calculate multiplier
        // apply limits
        // return result
    }
}
```

24. Polygon Service (point → polygon)

```
public interface PolygonService {
    Optional<ServicePolygon> findContainingPolygon(double latitude, double longitude);
}
```

Demand Aggregation Service (orders → polygon demand)

```
public interface DemandAggregationService {
    PolygonDemand getDemand(UUID polygonId);
}
```

Supply Aggregation Service (drivers → polygon supply)

```
public interface SupplyAggregationService {
    PolygonSupply getSupply(UUID polygonId);
}
```

25. Surge Scoring Service (signals → normalized score)

```
public interface SurgeScoringService {
    double calculateScore(SurgeFeatures features);
}
```

26. Surge Orchestrator

<pre>SurgePricingService +-- PolygonService +-- DemandService +-- SupplyService +-- ETAService +-- TrafficService +-- SurgeScoringService +-- PricingService</pre>	Surge Request <pre>public record SurgeRequest(double latitude, double longitude, UUID storeId, BigDecimal baseDeliveryFee) {}</pre> Surge Response <pre>public record SurgeResult(UUID polygonId, double pressureScore, double surgeMultiplier, BigDecimal surgeAmount, BigDecimal finalDeliveryFee, Instant expiresAt) {}</pre>
--	--

27. Real-Time Surge Updates

Every 30 seconds → Calculate polygon state → Update surge

28. Surge Heatmap

The frontend can render:

Polygon A → Normal	+-----+-----+
Polygon B → Medium	HIGH
Polygon C → High	NORMAL
Polygon D → Very High	+-----+-----+
	MEDIUM VERY
	HIGH
	+-----+-----+

29. ML-Based Surge Prediction

Instead of reacting only to: `current demand`

the system can predict:: `expected demand in 15 minutes`